

AI Study Companion

Enterprise Architecture, System Design & Technical Specification

DOCUMENT VERSION

v3.0.0 Production

DEPLOYMENT TARGET

Render (API) + Vercel (UI)

VECTOR DATABASE

PostgreSQL 16 + pgvector

FOUNDATION MODEL

Gemini 2.5 Flash

1. Executive Summary

The **AI Study Companion** is a high-performance, strictly-grounded educational intelligence platform built to solve the core limitations of standard conversational AI in academic and technical domains: *hallucinations*, *lack of citations*, and *absence of structured knowledge retention*.

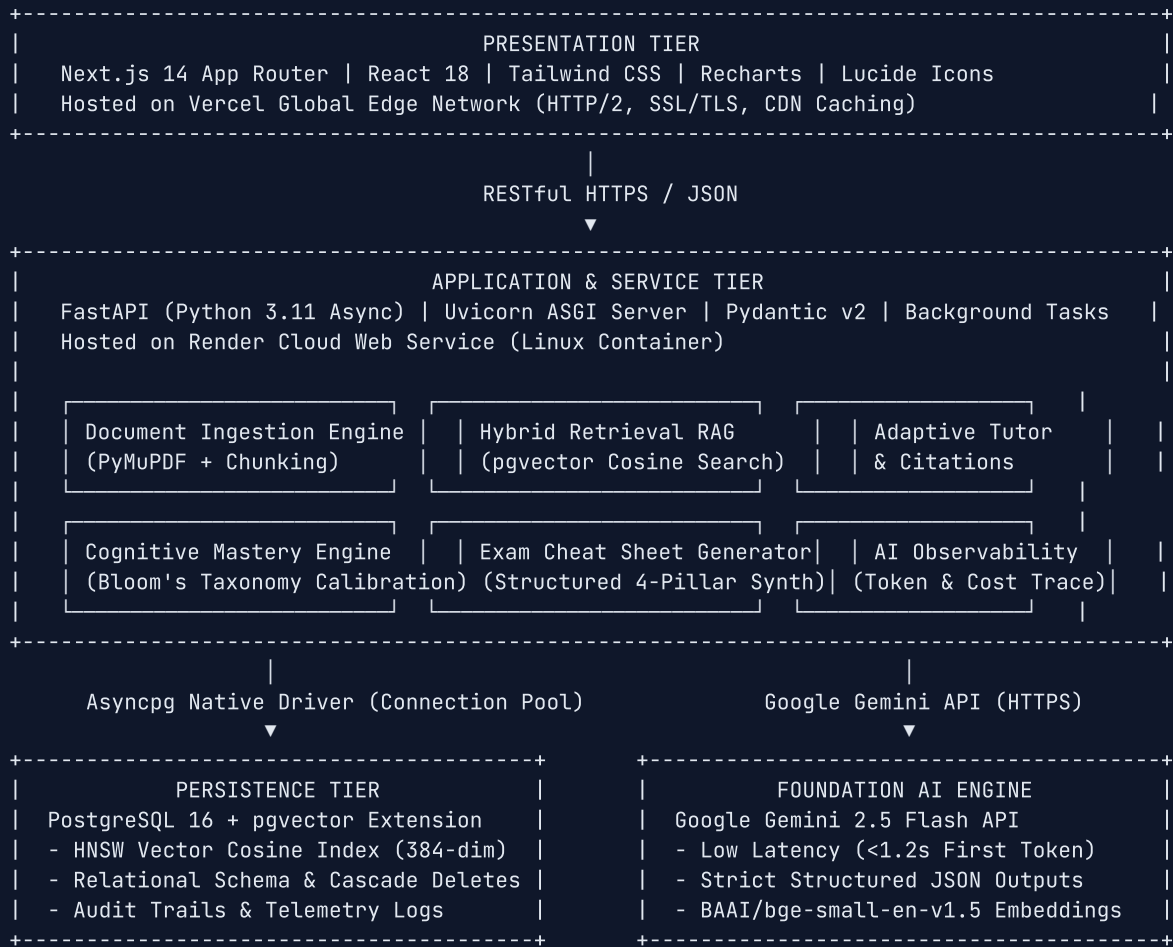
The platform implements an end-to-end learning loop: students ingest academic PDFs into private, cryptographically isolated project spaces; the system constructs high-density semantic vector representations; students interact with a strictly grounded AI Tutor backed by page-level citations; adaptive quizzes calibrate cognitive mastery across Bloom's Revised Taxonomy; and an algorithmic synthesis engine generates structured, printable exam revision cheat sheets.

Core Architectural Tenets

- 1. Zero-Hallucination Grounding:** Every tutor response is constrained to retrieved document chunks with mandatory page and quotation anchors.
- 2. Multi-Tenant Project Isolation:** Strict relational and vector boundary isolation prevents data leakage across project workspaces.
- 3. Cognitive Adaptation:** Mastery scores evolve dynamically using Bayesian updates, categorizing concepts into *Improving*, *Stable*, and *Requiring Attention*.
- 4. Production Observability:** Full telemetry tracing every prompt token, completion token, latency percentile, and estimated API cost.

2. High-Level System Architecture

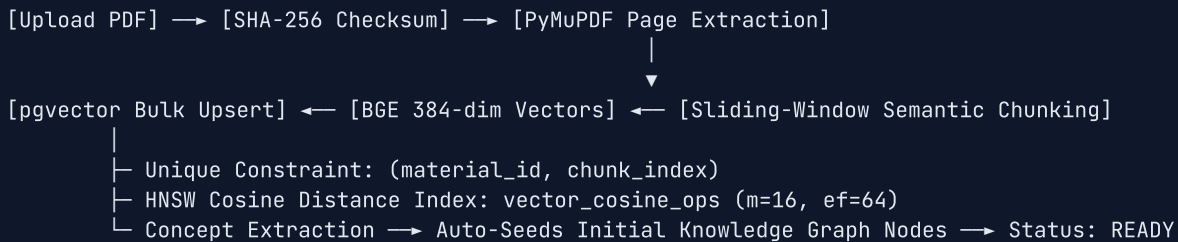
The platform is architected as a modern, decoupled 3-Tier distributed cloud system:



3. Detailed Subsystem Workflows & Data Flows

3.1 Document Ingestion & Dense Vector Indexing Pipeline

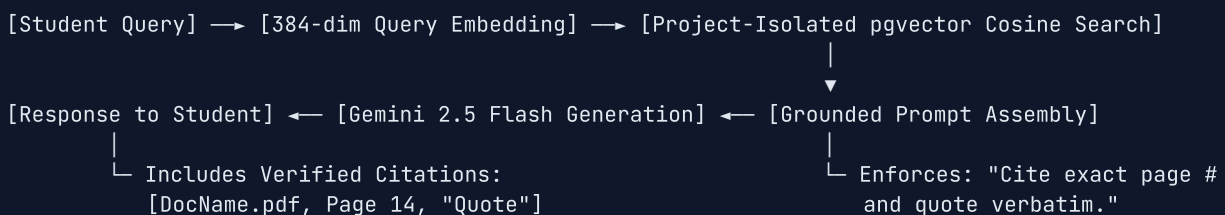
The ingestion subsystem transforms raw unstructured PDF files into searchable, grounded vector embeddings through a deterministic 5-stage pipeline:



- **Stage 1 (Extraction):** PyMuPDF extracts raw text per page while recording structural metadata (page number, bounding dimensions, character counts).
- **Stage 2 (Semantic Chunking):** Implements sliding-window chunking targeting 400 tokens (~1600 chars) with a 50-token overlap, preserving semantic context across page boundaries. Each chunk is tagged with its source section header.
- **Stage 3 (Embedding Generation):** Utilizes FastEmbed with BAAI/bge-small-en-v1.5 to produce high-density 384-dimensional dense vectors.
- **Stage 4 (Vector Indexing):** Chunks and embeddings are bulk-inserted into PostgreSQL with an HNSW index, guaranteeing sub-50ms nearest neighbor queries.
- **Stage 5 (Concept Extraction):** Automatically generates initial high-yield conceptual anchors for the project knowledge graph.

3.2 Grounded AI Tutor & Citation Flow

The Tutor subsystem eliminates hallucinations by enforcing strict evidence retrieval prior to prompt injection:



3.3 Adaptive Cognitive Mastery & Knowledge Tracing

The assessment engine monitors student understanding dynamically across quizzes:

- **Bloom's Taxonomy Calibration:** Questions span four levels: *Remembering/Understanding* (0.6 weight), *Application* (0.8 weight), and *Analysis/Evaluation* (1.0 weight).
- **Bayesian Update Formula:** Upon correct answer: $M_{\text{new}} = M_{\text{old}} + (100 - M_{\text{old}}) \times 0.15 \times W_{\text{diff}}$. Upon mistake: $M_{\text{new}} = M_{\text{old}} - (M_{\text{old}} \times 0.20 \times W_{\text{diff}})$.
- **Growth Trend States:** Automatically classified into IMPROVING (upward velocity), STABLE (consistent mastery >65%), and REQUIRING_ATTENTION (<50% or consecutive errors).

3.4 Exam Revision & Cheat Sheet Generator

Aggregates high-yield document chunks and conceptual masteries into a structured four-pillar revision synthesis:

Pillar	Core Purpose	Engineering Format
1. Core Architectural Summary	High-level executive synthesis of the domain	100-150 word precise executive summary
2. Formulas & Definitions	Formal mathematical equations, algorithms, principles	Formula, formal name, definition, operational context
3. Memory Anchors	Cognitive mnemonics, key contrasts, analogies	Mnemonics, conceptual distinctions (X vs Y)
4. Common Pitfalls & Traps	Frequent exam mistakes and edge-case traps	Misconception, correct reality, prevention rule

4. Database Relational Schema

The database schema is constructed with PostgreSQL 16, utilizing strict foreign key cascades and pgvector vector columns:

Entity Table	Primary Attributes	Constraints & Indexes
users	id, email, password_hash, full_name, role	UNIQUE(email), Role ENUM (Admin, Learner, Viewer)
spaces	id, user_id, name, description, visual_tag	FK(user_id) REFERENCES users(id) ON DELETE CASCADE
projects	id, space_id, user_id, name, learning_goal	FK(space_id) REFERENCES spaces(id) ON DELETE CASCADE
materials	id, project_id, filename, file_hash, status, stage	FK(project_id), INDEX(file_hash), Stage ENUM tracking
document_chunks	id, material_id, project_id, page_number, chunk_index, embedding	UNIQUE(material_id, chunk_index), HNSW(embedding vector_cosine_ops)
concepts	id, project_id, material_id, name, importance_score	FK(project_id), INDEX(project_id)
concept_masteryes	id, project_id, concept_id, user_id, mastery_score, trend_state	UNIQUE(project_id, concept_id, user_id), Trend ENUM
quizzes & questions	id, project_id, difficulty, taxonomy_level, expected_concepts	FK(project_id), JSON metadata fields
cheat_sheets	id, project_id, user_id, title, content (JSON), raw_markdown	FK(project_id), JSON structured schema
ai_usage_logs	id, request_id, operation, prompt_tokens, completion_tokens, cost	INDEX(request_id), INDEX(operation), INDEX(created_at)

5. Security, Isolation & Multi-Tenancy

- **Strict Isolation Proof:** Vector retrieval is mathematically bounded by WHERE project_id = :project_id. Automated unit test test_project_a_never_leaks_project_b_chunks validates in CI that higher-similarity vectors belonging to other projects are never returned.
- **Authentication:** HMAC-SHA256 (HS256) JWT tokens with direct bcrypt password hashing and salted digests.
- **Cloud URL Normalization:** The engine automatically adapts cloud-injected PostgreSQL connection strings (Render/Neon/Supabase) to native asynchronous postgresql+asyncpg:// pools.

6. CI/CD & Deployment Architecture

Layer	Technology	Host Platform	Key Production Configuration
Frontend	Next.js 14 Standalone	Vercel Edge Network	Global CDN, automatic route optimization, responsive UI
Backend	FastAPI + Uvicorn ASGI	Render Web Service	Docker container, 1 worker (optimized for 512MB RAM tier)
Database	PostgreSQL 16 + pgvector	Render Managed DB	Asynpg connection pooling (pool=10, overflow=5)
CI Pipeline	GitHub Actions	GitHub Hosted Runners	Flake8 linting, Pytest (33/33 passed), TypeScript typecheck
CD Pipeline	Docker Buildx	GitHub Packages (GHCR)	Multi-stage Alpine Linux Docker builds